CRUD Operations in MongoDB

MongoDB Cluster, DB, Collection, Document

The following are descriptions of the relationship between clusters, databases, collections, and documents in MongoDB:

1. Cluster: A cluster in MongoDB is a group of servers that work together to provide high availability, fault tolerance, and scalability for your data. A cluster can consist of one or more physical servers, each running an instance of the MongoDB database.

2. Database: A database is a logical container for collections of documents. Each database in MongoDB can contain multiple collections, and each collection can have its own schema and indexes.

3. Collection: A collection is a group of documents that share a common schema. Collections in MongoDB are analogous to tables in a relational database, but with a more flexible schema. Collections can be created dynamically and can contain documents with different fields.

4. Document: A document is a set of key-value pairs that represent the data to be stored in a collection. Each document in MongoDB has a unique identifier (_id) and can contain nested documents and arrays.

Below is a table of the MongoDB concepts of cluster, database, collection, and document with their counterparts in a relational database:

MongoDB/NoSQL	Relational Database
Cluster	Database Server/Instance (DB_jo)
Database	Database (bludb)
Collection	Table (sssales)
Document	Row/Record

MongoDB is a document-oriented database that stores data in a flexible, schema-free format. Clusters are groups of servers that work together to provide high availability and scalability for your data, databases are logical containers for collections of documents, collections are groups of documents with a common schema, and documents are sets of key-value pairs that represent the data to be stored.

Create a database

To create a database in MongoDB, you can use the use command followed by the name of the database you want to create. Here's an example:

```
use <database-name>
```

If the specified database does not already exist, MongoDB will create it for you. Note that the database will not be created until you insert some data into it.

Alternatively, you can also create a database using the db.createCollection() method. This method creates a new collection with the specified name in the specified database, and if the database does not exist, it creates the database as well. Here's an example:

```
db.createCollection("<collection-name")
```

Drop a database

To drop a database in MongoDB, you can use the dropDatabase() method. Here's the syntax:

```
use <database_name>
db.dropDatabase()
```

In this syntax, replace <database_name> with the name of the database you want to drop. For example, if you want to drop a database called "mydatabase", you would do the following:

```
use mydatabase
db.dropDatabase()
```

This will drop the "mydatabase" database and permanently delete all of its data. Note that this operation cannot be undone, so be careful when using this command.

Create a collection in an alternate way

In MongoDB, you can create a collection implicitly by inserting a document into it. When you insert a document into a non-existent collection, MongoDB automatically creates the collection for you.

Here's an example:

```
use mydatabase
db.mycollection.insertOne({ name: "John", age: 30 })
```

In this example, we first switch to the "mydatabase" database using the use command. Then, we insert a document with the fields "name" and "age" into a collection called "mycollection". If "mycollection" does not exist, MongoDB will create it automatically.

Note that this method of creating a collection implicitly may not be suitable for all scenarios, especially if you need to set specific options or indexes for the collection. In such cases, you should use the createCollection() method to create the collection explicitly.

Delete a collection

To delete a collection in MongoDB, you can use the drop() method on the collection object. Here's the syntax:

```
db.<collection_name>.drop()
```

In this syntax, replace <collection_name> with the name of the collection you want to delete.

For example, if you want to delete a collection called "mycollection", you would do the following:

```
db.mycollection.drop()
```

This will delete the "mycollection" collection and permanently remove all of its data. Note that this operation cannot be undone, so be careful when using this command.

Create a sample collection

To create a collection called "neu" with the fields "lastName", "firstName", "program", and "age", with a value of "Dela Cruz", "Juan", "BSIT", and 21 respectively, you can use the insertOne() method to insert a document into the collection with the desired field values. Here's an example:

```
db.neu.insertOne({ lastName: "Dela Cruz", firstName: "Juan", program: "BSIT", age: 21 })
```

In this example, we are creating a new collection called "neu" and inserting a document into it using the insertOne() method. The document has the fields "lastName", "firstName", "program", and "age" with the values "Dela Cruz", "Juan", "BSIT", and 21 respectively.

Note that by default, MongoDB stores all numeric values as 64-bit floating-point numbers, so the "age" field will be stored as a floating-point number unless you explicitly declare it as an integer. To declare the "age" field as an integer, you can use the $numberInt operator. Here's an example:

```
db.neu.insertOne({ lastName: "Dela Cruz", firstName: "Juan", program: "BSIT", age: { $numberInt: "21" } })
```

In this example, we are using the $numberInt operator to declare the "age" field as an integer with a value of 21.

Data Types in MongoDB

In MongoDB, there are several data types that can be used to store data in documents. Below are the brief descriptions of each data type:

1. String: A string data type represents a sequence of characters that can be Unicode. Strings are commonly used to store textual data.

2. Integer: An integer data type represents a whole number without a decimal point. Integers can be used to store values such as age, quantity, or any other whole number.

3. Double: A double data type is used to represent a floating-point number with decimal places. Doubles are commonly used to store values such as weight, height, or any other value that can have decimal places.

4. Boolean: A boolean data type represents a logical value of true or false. Booleans are commonly used to represent the state of a condition or decision.

5. Date: A date data type represents a date and time value. Dates are commonly used to store information such as when a document was created or when an event occurred.

6. Object ID: An object ID data type represents a unique identifier for a document. MongoDB automatically generates an object ID for each document created in a collection.

7. Array: An array data type represents a collection of values, which can be of any data type. Arrays are commonly used to store a list of items, such as interests or addresses.

8. Null: A null data type represents a non-existent or unknown value. Nulls are commonly used to indicate that a field has not been set or that the value is unknown.

9. Undefined: An undefined data type represents a field that has not been defined or does not exist in a document.

10. Binary: A binary data type represents binary data, such as images or files.

11. Regular expression: A regular expression data type represents a pattern used to match strings. Regular expressions are commonly used to search and manipulate textual data.

Overall, MongoDB provides a flexible and dynamic schema that allows for the use of various data types to store data in documents.

Understand the _id field

In MongoDB, the _id field is a unique identifier that is automatically generated for each document in a collection. It serves as the primary key for the document and is used by MongoDB to ensure that each document in a collection is unique.

By default, the _id field is an ObjectId, which is a 12-byte hexadecimal value that includes a timestamp, a machine identifier, and a random value. The timestamp portion of the ObjectId makes it possible to sort documents in the order they were created.

You can also specify a value for the _id field when inserting a document into a collection. If the value you specify is not unique, MongoDB will throw a duplicate key error.

The _id field is automatically indexed by MongoDB, which makes it efficient for querying documents by their unique identifier. You can also use the _id field in aggregation pipelines to group, sort, and filter documents.

It's important to note that the _id field is unique within a collection, but not across multiple collections or databases. If you need to reference a document from another collection or database, you can use the _id value as a foreign key.

How to understand find() command

The **find()** command is a basic query method in MongoDB that retrieves documents from a collection based on a set of criteria. The syntax for the find() command is as follows:

```
db.collection.find(query, projection)
```

Here, collection refers to the name of the collection you want to query, query is an optional parameter that specifies the criteria for selecting documents, and projection is an optional parameter that specifies the fields to include or exclude from the returned documents.

The query parameter is typically specified using a set of key-value pairs that represent the criteria for selecting documents. For example, if you want to retrieve all documents where the lastName field is "Dela Cruz", you can use the following command:

db.neu.find({ lastName: "Dela Cruz" })

This will retrieve all documents from the neu collection where the lastName field is equal to "Dela Cruz".

The pretty() in find

In MongoDB, the pretty() method is used to format the output of a find() query in a more readable and organized way. When you append the pretty() method to a find() query, it will return the query results in a structured JSON format that is easier to read and understand.

For example, suppose you have a collection called users with the following documents:

```
{ "_id": 1, "name": "Alice", "age": 25 }
{ "_id": 2, "name": "Bob", "age": 30 }
{ "_id": 3, "name": "Charlie", "age": 35 }
```

If you run the following find() query on the users collection:

```
db.users.find()
```

The output will be a single line with no formatting:

```
{ "_id": 1, "name": "Alice", "age": 25 }{ "_id": 2, "name": "Bob", "age": 30 }{ "_id": 3, "name": "Charlie", "age": 35 }
```

However, if you append the pretty() method to the query like this:

```
db.users.find().pretty()
```

The output will be formatted with line breaks and indentation, making it easier to read:

```
{
	"_id" : 1,
	"name" : "Alice",
	"age" : 25
}
{
	"_id" : 2,
```

```
        "name" : "Bob",
        "age" : 30
    }
    {
        "_id" : 3,
        "name" : "Charlie",
        "age" : 35
    }
```

Using pretty() is especially helpful when dealing with large or complex queries that return multiple documents. It can help make the results more readable and easier to work with.

Insert document

Here is the command shell code that will create each document with the specified fields and values in the neu collection, and display the document using the find() command:

```
>db.neu.insertOne({lastName: 'Santos', firstName: 'Maria', program:
'BSIS', age: 19})
>db.neu.insertOne({lastName: 'Madrigal', firstName: 'Alex', program:
'BSEMC', age: 20})
```

Display the documents:

```
> db.neu.find()
[
 {
  _id: ObjectId("64464ab405791bc0039ff2ba"),
  lastName: 'Dela Cruz',
  firstName: 'Juan',
  program: 'BSIT',
  age: 21
 },
 {
  _id: ObjectId("64464b1405791bc0039ff2bb"),
  lastName: 'Santos',
  firstName: 'Maria',
  program: 'BSIS',
  age: 19
 },
 {
  _id: ObjectId("64464b4205791bc0039ff2bc"),
  lastName: 'Madrigal',
```

```
    firstName: 'Alex',
    program: 'BSEMC',
    age: 20
  }
]
```

The 2 commands executed each document in the neu collection with the specified fields and values using the insertOne() method. The find() method without any arguments to display all documents in the neu collection, which should include the newly created document.

Insert many documents

You can use the insertMany() method to insert multiple documents at once into the neu collection with the specified fields and values. Here's the command shell code to insert the two documents you provided:

```
>db.neu.insertMany([{lastName: 'Contreras', firstName: 'Shane',
program: 'BSIT', age: 22}, {lastName: 'Malabanan', firstName: 'Jinx',
program: 'BSCS', age: 18}])

{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId("64464dc005791bc0039ff2be"),
    '1': ObjectId("64464dc005791bc0039ff2bf")
  }
}
```

You can verify that the documents have been inserted by running the find() method:

```
> db.neu.find()
[
  {
    _id: ObjectId("64464ab405791bc0039ff2ba"),
    lastName: 'Dela Cruz',
    firstName: 'Juan',
    program: 'BSIT',
    age: 21
  },
  {
    _id: ObjectId("64464b1405791bc0039ff2bb"),
    lastName: 'Santos',
```

```
    firstName: 'Maria',
    program: 'BSIS',
    age: 19
  },
  {
    _id: ObjectId("64464b4205791bc0039ff2bc"),
    lastName: 'Madrigal',
    firstName: 'Alex',
    program: 'BSEMC',
    age: 20
  },
  {
    _id: ObjectId("64464dc005791bc0039ff2be"),
    lastName: 'Contreras',
    firstName: 'Shane',
    program: 'BSIT',
    age: 22
  },
  {
    _id: ObjectId("64464dc005791bc0039ff2bf"),
    lastName: 'Malabanan',
    firstName: 'Jinx',
    program: 'BSCS',
    age: 18
  }
]
```

Update document

You can use the updateMany() method to update all documents in the neu collection where the program field is equal to 'BSIT' and replace it with the new value 'BS Information Technology'. Here's the command shell code to do that:

```
>  db.neu.updateMany(
... { program: 'BSIT'},
... { $set :
... { program: 'BS Information Technology'}
... }
... )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
```

```
  modifiedCount: 1,
  upsertedCount: 0
}
```

The above command updates all documents in the neu collection where the program field is equal to 'BSIT' and sets it to 'BS Information Technology'. The $set operator is used to specify the new value of the program field.

```
> db.neu.find()
[
  {
    _id: ObjectId("64464ab405791bc0039ff2ba"),
    lastName: 'Dela Cruz',
    firstName: 'Juan',
    program: 'BS Information Technology',
    age: 21
  },
  {
    _id: ObjectId("64464b1405791bc0039ff2bb"),
    lastName: 'Santos',
    firstName: 'Maria',
    program: 'BSIS',
    age: 19
  },
  {
    _id: ObjectId("64464b4205791bc0039ff2bc"),
    lastName: 'Madrigal',
    firstName: 'Alex',
    program: 'BSEMC',
    age: 20
  },
  {
    _id: ObjectId("64464dc005791bc0039ff2be"),
    lastName: 'Contreras',
    firstName: 'Shane',
    program: 'BS Information Technology',
    age: 22
  },
  {
    _id: ObjectId("64464dc005791bc0039ff2bf"),
    lastName: 'Malabanan',
    firstName: 'Jinx',
    program: 'BSCS',
    age: 18
```

```
 }
]
```

Delete documents

To delete all documents in the neu collection where the program field contains the value 'BS Information Technology'. The filter object passed to deleteMany() simply specifies that the program field must have the value 'BS Information Technology'.

```
> db.neu.deleteMany(
... { program: 'BS Information Technology'}
... )
```

{ acknowledged: true, deletedCount: 2 }

Display the update collection:

```
> db.neu.find()
[
 {
  _id: ObjectId("64464b1405791bc0039ff2bb"),
  lastName: 'Santos',
  firstName: 'Maria',
  program: 'BSIS',
  age: 19
 },
 {
  _id: ObjectId("64464b4205791bc0039ff2bc"),
  lastName: 'Madrigal',
  firstName: 'Alex',
  program: 'BSEMC',
  age: 20
 },
 {
  _id: ObjectId("64464dc005791bc0039ff2bf"),
  lastName: 'Malabanan',
  firstName: 'Jinx',
  program: 'BSCS',
  age: 18
 }
]
```

How to insert field in the existing collection

To insert a new field weight with a double data type to an existing neu collection in MongoDB, you can use the updateMany() command with an empty filter object {} to apply the update operation to all documents in the collection. Here's an example command shell code:

```
> db.neu.updateMany(
... {},
... {$set :
... { weight: 0.1}
... }
... )
```

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 3,
  modifiedCount: 3,
  upsertedCount: 0
}
```

This will add the new field weight to all documents in the neu collection and set its value to 0.1. The $set operator is used to set the value of the weight field. The {} filter object matches all documents in the collection so the update operation will be applied to all documents.

As you display the updated collection:

```
> db.neu.find()
[
  {
    _id: ObjectId("64464b1405791bc0039ff2bb"),
    lastName: 'Santos',
    firstName: 'Maria',
    program: 'BSIS',
    age: 19,
    weight: 0
  },
  {
    _id: ObjectId("64464b4205791bc0039ff2bc"),
    lastName: 'Madrigal',
    firstName: 'Alex',
    program: 'BSEMC',
    age: 20,
    weight: 0
```

```
  },
  {
    _id: ObjectId("64464dc005791bc0039ff2bf"),
    lastName: 'Malabanan',
    firstName: 'Jinx',
    program: 'BSCS',
    age: 18,
    weight: 0
  }
]
```

How to delete field from collection

To delete the existing field "weight" from the "neu" collection using the updateMany command, you can use the $unset operator. The syntax for the updateMany command would be:

```
> db.neu.updateMany(
... {},
... {$unset:
... { weight:""}
... }
... )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 3,
  modifiedCount: 3,
  upsertedCount: 0
}

> db.neu.find()
[
  {
    _id: ObjectId("64464b1405791bc0039ff2bb"),
    lastName: 'Santos',
    firstName: 'Maria',
    program: 'BSIS',
    age: 19
  },
  {
    _id: ObjectId("64464b4205791bc0039ff2bc"),
    lastName: 'Madrigal',
```

```
    firstName: 'Alex',
    program: 'BSEMC',
    age: 20
  },
  {
    _id: ObjectId("64464dc005791bc0039ff2bf"),
    lastName: 'Malabanan',
    firstName: 'Jinx',
    program: 'BSCS',
    age: 18
  }
]
```

Insert an array field in the existing collection

To add an array field named "interest" to an existing "neu" collection, you can use the updateMany command with $set and $push operators:

```
>db.neu.updateMany({}, {$set: {interest: []}});
```

The above command will add an empty array field named "interest" to all the documents in the "neu" collection.

To add interests for a specific document, you can use the $push operator along with the updateOne command. For example, to add interests "programming" and "reading" for a document where firstName is "Juan", you can use the following command:

```
>db.neu.updateOne(
... { firstName: 'Maria' },
... { $push:
... { interest :
... { $each :
... [ 'programming', 'reading']
... }
... }
... }
... )
```

The above command will add the values "programming" and "reading" to the "interest" array field of the document where the "firstName" field is "Juan". If the "interest" field does not exist in the document, it will be created with the values "programming" and "reading".

> db.neu.find()

```
[
  {
    _id: ObjectId("64474907698f3da2eb4cb382"),
    lastName: 'Santos',
    firstName: 'Maria',
    program: 'BSIS',
    age: 19,
    interest: [ 'programming', 'reading' ]
  },
  {
    _id: ObjectId("64474921698f3da2eb4cb383"),
    lastName: 'Madrigal',
    firstName: 'Alex',
    program: 'BSEMC',
    age: 20,
    interest: []
  },
  {
    _id: ObjectId("64474a3e698f3da2eb4cb385"),
    lastName: 'Malabanan',
    firstName: 'Jinx',
    program: 'BSCS',
    age: 18,
    interest: []
  }
]
```

Insert an object field

To create an object in the MongoDB shell, you can simply declare it as a JavaScript object literal. Here's an example of how you can create an object and attach it to a customer document:

```
// Create a customer document with an object field
>db.customer.insertOne(
... { customerName: 'New Era University', address :
... { street: '9 Central Ave.', city: 'Quezon City', province: 'NCR'}
... }
... )
{
  acknowledged: true,
  insertedId: ObjectId("644750a6698f3da2eb4cb386")
```

}

In this example, we created a customer document with a nested object field called address. The address object contains several key-value pairs, which represent the street, city, state, and zip code of the customer's address.

Display the content:

```
>db.customer.find().pretty()
[
  {
    _id: ObjectId("644750a6698f3da2eb4cb386"),
    customerName: 'New Era University',
    address: { street: '9 Central Ave.', city: 'Quezon City', province: 'NCR' }
  }
]
```

Note that MongoDB supports objects with nested objects, arrays, and other complex structures.